

DEPARTAMENTO
DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

Trabajo practico 2C 2026

Especificación de TAD: Sistema de Fidelización AirMiles

September 14, 2026 Algoritmos y Estructuras de Datos

MaNaLeTo

Integrante	LU	Correo electrónico
Alonso, Nahuel Nicolas	692/25	alonsonahuel35@gmail.com
Carrasco, Tomás Ezequiel	663/25	tomásezequielcarrasco@gmail.com
Gatica, Mauricio Nahuel	1198/24	gaticamauricio2308@gmail.com
Tortarolo, Leon	1363/25	leontortarolo1@gmail.com

Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires
Ciudad Universitaria - (Pabellón I/Planta Baja)
Intendente Güiraldes 2610 - C1428EGA
Ciudad Autónoma de Buenos Aires - Rep. Argentina
Tel/Fax: (++54 +11) 4576-3300
http://www.exactas.uba.ar

1 Enunciado

En el competitivo mundo de la aviación, las aerolineas buscan constantemente fidelizar a sus pasajeros mediante programás de millas. AirMiles es un sistema que permite a los usuarios acumular millas por cada vuelo realizado, canjearlas por pasajes o mejoras, y transferirlas a otros usuarios. Además, los pasajeros pertenecen a diferentes Categorías (Bronce, Plata, Oro, Platino) que determinan la cantidad de millas que ganan por kilometro volado. Para incentivar el crecimiento de la base de usuarios, se implementa un sistema de referidos: cuando un nuevo usuario se registra gracias a la recomendación de otro, ambos reciben un bono de 500 millas.

Se necesita un sistema que gestione de manera confiable los saldos de millas, asegurando que no se puedan gastar millas que no se poseen y que las Categorías se actualicen correctamente en función del total historico de millas voladas.

El sistema debe almacenar el historial de transacciones de cada usuario, incluyendo vuelos realizados, millas ganadas, millas canjeadas y transferencias realizadas. Esto permitira a los usuarios consultar su historial y a la aerolinea analizar el comportamiento de los pasajeros. El sistema también debe ser capaz de proveer el saldo de millas disponibles para cada usuario, asi como el total acumulado a lo largo del tiempo. Se debe cumplir que para cada usuario, las millas disponibles no superen las totales.

El sistema debe permitir conocer la categoría actual de cada usuario que se calcula según el total de millas acumuladas. Los usuarios que llegan al millon de millas acumuladas alcanzan la categoría Platino, que les otorga beneficios exclusivos, pero ya no acumulan más millas.

Algunos aspectos interesantes del sistema incluyen la posibilidad de Promociónes especiales que aumenten temporalmente el factor de acumulación de millas. Además, la aerolinea podria querer implementar un ranking de usuarios con más millas disponibles para ofrecer beneficios exclusivos.

2 Especificación

2.1 Structs

Id ES $\mathbb{Z}$
Usuarios ES dict$\langle$Id, Historial$\rangle$

Millas, MillasAcumuladas, MillasCanjeadas, MillasTransferidas, MillasTransferidasEnviadas, MillasTransferidasRecibidas, MillasReferidas ES $\mathbb{R}$

enum Categoría ES {Bronce, Plata, Oro, Platino}

Historial ES struct$\langle$*acumular* : seq$\langle$TransacciónAcumular$\rangle$, *canjes* : seq$\langle$TransacciónCanje$\rangle$, *transferencias* : seq$\langle$TransacciónTransferencia$\rangle$, *referencias* : seq$\langle$TransacciónReferencia$\rangle\rangle$

TransacciónAcumular ES struct$\langle$*promociónAplicada* : string, *factorPromoAplicada* : $\mathbb{R}$, *millasObtenidas* : Millas, *distanciaVolada* : $\mathbb{R}\rangle$

TransacciónCanje ES struct$\langle$*precioEnMillas* : Millas, *premioCanjeado* : string$\rangle$

TransacciónTransferencia ES struct $\langle$*idOrigen* : Id, *idDestino* : Id, *millasTransferidas* : MillasTransferidas$\rangle$

TransacciónReferencia ES struct$\langle$*idReferidor* : Id, *idReferido* : Id, *millasRecibidas* : Millas$\rangle$

/*En nuestro sistema consideramos que una referencia también es un tipo de transacción por lo que definimos su estructura para poder registrarla en el historial del usuario*/

Promo ES struct$\langle nombre : \mathsf{string}, factor : \mathbb{R}, tope : \mathsf{conj}\langle\mathbb{R}\rangle\rangle$

/*La variable tope dentro de Promo indica tanto la existencia del tope como su valor, siendo un conjunto vacío si se trata de una promoción sin tope y un conjunto de un único número real en caso de ser una promoción con tope*/

2.2 TAD

TAD AirMiles {

obs usuarios : Usuarios

obs promo : Promo

proc crearSistema() : AirMiles {

requiere { $True$ }

asegura { $esPromoPredeterminada(res.promo) \land \neg hayUsuarios(res)$ }

}

proc registrarUsuario(inout $airMiles$: AirMiles) {

requiere { $A_0 = airMiles$ }

asegura {

$mismaPromo(A_0, airMiles)$

$\land\, usuariosViejosNoCambiaron(A_0, airMiles)$

$\land\, tieneUnSoloUsuarioMás(A_0, airMiles)$

$\land\, losUsuariosNuevosTienenHistorialVacío(A_0, airMiles)$

}

}

pred usuariosViejosNoCambiaron($viejo$: AirMiles, $nuevo$: AirMiles) {

$(\forall id : \mathsf{Id})$ (

$esUsuarioDe(id, viejo) \rightarrow esUsuarioDe(id, nuevo)$

$\land_L viejo.usuarios[id] = nuevo.usuarios[id]$

)

}

pred losUsuariosNuevosTienenHistorialVacío($viejo$: AirMiles, $nuevo$: AirMiles) {

$(\forall id : \mathsf{Id})$ $(esUsuarioNuevo(id, viejo, nuevo) \rightarrow_L historialVacío() = nuevo.usuarios[id])$

}

proc registrarUsuarioConReferidor(in $idReferidor$: Id, inout $airMiles$: AirMiles) {

requiere { $esUsuarioDe(idReferidor, airMiles) \land A_0 = airMiles$ }

asegura {

$mismaPromo(A_0, airMiles)$

$\land\, tieneUnSoloUsuarioMás(A_0, airMiles)$

$\land\, usuariosIgualesExcepto(A_0, airMiles, \langle idReferidor\rangle)$

$\land\, (\forall idReferido : \mathsf{Id})$ (

$esUsuarioNuevo(idReferido, A_0, airMiles) \rightarrow_L$

$seAgregaronTransaccionesPorReferir(idReferidor, idReferido, A_0, airMiles)$

)

```
    }
}

pred seAgregaronTransaccionesPorReferir(idReferidor : Id, idReferido : Id, viejo : AirMiles,
nuevo : AirMiles) {
    registróReferencia(viejo.usuarios[idReferidor], nuevo.usuarios[idReferidor],
    idReferidor, idReferido)
    ∧ historialesIgualesExceptoReferencia(historialVacío(), nuevo.usuarios[idReferido])
    ∧ nuevo.usuarios[idReferido].referencias = seq⟨referenciaNueva(idReferidor, idReferido)⟩
}

pred registróReferencia(viejo : Historial, nuevo : Historial, idReferidor : Id, idReferido : Id) {
    nuevo.referencias = viejo.referencias + +referenciaNueva(idReferidor, idReferido)
    ∧ historialesIgualesExceptoReferencia(viejo, nuevo)
}

aux referenciaNueva(idReferidor : Id, idReferido : Id) : TransacciónReferencia =
⟨idReferidor : idReferidor, idReferido : idReferido, millasRecibidas : 500⟩

/*Interpretamos que el tope de la promoción indica la cantidad máxima de millas que aporta al total.
Al superarlo, simplemente sumamos la distancia multiplicada por el factor de la categoría del usuario. Es
decir, se utiliza parte de la distancia para alcanzar el tope de la promoción y al resto se le aplica un factor
de 1*/


proc acumularMillas(inout airMiles : AirMiles, in distancia : ℝ, in id : Id) {
   requiere {
       esUsuarioDe(id, airMiles)
       ∧ distancia > 0
       ∧ A0 = airMiles
   }
   asegura {
       mismaCantidadDeUsuarios(A0, airMiles)
       ∧ usuariosIgualesExcepto(A0, airMiles, ⟨id⟩)
       ∧ esUsuarioDe(id, airMiles)
       ∧L (∃factorCategoría : ℝ) (
           esFactorDe(factorCategoría, millasTotales(A0.usuarios[id]))
           ∧(∃millas : Millas) (
               millas = IfThenElse(tieneTope(A0.promo),
               millasConTope(distancia, factorCategoría, id, A0),
               millasSinTope(distancia, factorCategoría, A0))
               ∧restóDeTopeSiTieneYCanceló SiCero(A0.promo, airMiles.promo, millas)
               ∧noModificóLaPromoSiNoTieneTope(A0.promo, airMiles.promo)
               ∧registróAcumulación(A0.usuarios[id], airMiles.usuarios[id],
               millas, distancia, A0.promo)
           )
       )
   }
}
```

aux millasConTope($distancia : \mathbb{R}, factorCategoría : \mathbb{R}, id$: Id, $airMiles$: AirMiles) : Millas =
IfThenElse($superaElTope(distancia, factorCategoría, id, airMiles)$,
$millasSuperadoElTope(topeDePromoConTope(airMiles.promo), factorCategoría,$
$distancia, airMiles.promo.factor)$,
$millasSinTope(distancia, factorCategoría, airMiles)$)

pred superaElTope($distancia : \mathbb{R}, factorCategoría : \mathbb{R}, id$: Id, $airMiles$: AirMiles) {
$topeDePromoConTope(airMiles.promo) < millasSinTope(distancia, factorCategoría, airMiles)$
}

aux millasSuperadoElTope($tope$: Millas, $factorCategoría : \mathbb{R}, distancia : \mathbb{R}, factorPromo : \mathbb{R}$) : Millas =
tope+factorCategoría*(distancia-tope/(factorPromo*factorCategoría))

aux millasSinTope($distancia : \mathbb{R}, factorCategoría : \mathbb{R}, airMiles$: AirMiles) : Millas =
$distancia * factorCategoría * airMiles.promo.factor$

pred restóDeTopeSiTieneYCanceló SiCero($viejo$: Promo, $nuevo$: Promo, $millas$: Millas) {
$tieneTope(viejo) \rightarrow ($
$(millas \geq topeDePromoConTope(viejo) \rightarrow esPromoPredeterminada(nuevo))$
$\wedge(millas < topeDePromoConTope(viejo) \rightarrow mismoNombreFactorRestóDeTope(viejo, nuevo, millas)))$
}

pred mismoNombreFactorRestóDeTope($viejo$: Promo, $nuevo$: Promo, $millas$: Millas) {
$nuevo.tope = \langle topeDePromoConTope(viejo) - millas \rangle$
$\wedge\, nuevo.nombre = viejo.nombre$
$\wedge\, nuevo.factor = viejo.factor$
}

pred noModificóLaPromoSiNoTieneTope($viejo$: Promo, $nuevo$: Promo) {
$\neg tieneTope(viejo) \rightarrow nuevo = viejo$
}

pred registróAcumulación($viejo$: Historial, $nuevo$: Historial, $millasAcumuladas$: MillasAcumuladas,
$distancia : \mathbb{R}, promo$: Promo) {
$nuevo.acumular = viejo.acumular + +transacciónAcumularNueva(millasAcumuladas,$
$distancia, promo)$
$\wedge\, historialesIgualesExceptoAcumular(viejo, nuevo)$
}

aux transacciónAcumularNueva($millasObtenidas$: Millas, $distancia : \mathbb{R}, promo$: Promo) : TransacciónAcumular
=

$\langle distanciaVolada : distancia,$
$millasObtenidas : millasObtenidas,$
$promociónAplicada : promo.nombre,$
$factorPromoAplicada : promo.factor \rangle$
proc canjearMillas(in id : Id, in $precioEnMillas$: Millas, in $premio$: string, inout $airMiles$: AirMiles) {
requiere {

$A_0 = airMiles$
$\wedge\, precioEnMillas > 0$
$\wedge\, esUsuarioDe(id, airMiles)$
$\wedge_L\, millasDisponibles(airMiles.usuarios[id]) \geq precioEnMillas$
}
asegura {
$mismaPromo(A_0, airMiles)$
$\wedge\, usuariosIgualesExcepto(A_0, airMiles, \langle id \rangle)$
$\wedge\, mismaCantidadDeUsuarios(A_0, airMiles)$
$\wedge\, esUsuarioDe(id, airMiles)$
$\wedge_L\, agregóCanjeAlHistorial(A_0.usuarios[id], airMiles.usuarios[id], precioEnMillas, premio)$
}
}

pred agregóCanjeAlHistorial($viejo$: Historial, $nuevo$: Historial, $precioEnMillas$: Millas, $premio$: string)
{
$nuevo.canjes = viejo.canjes + +transacciónCanjeNueva(precioEnMillas, premio)$
$\wedge\, historialesIgualesExceptoCanjes(viejo, nuevo)$
}

aux transacciónCanjeNueva($precioEnMillas$: Millas, $premio$: string) : TransacciónCanje =
$\langle precioEnMillas : precioEnMillas, premioCanjeado : premio \rangle$

proc transferirMillas(in $millasTransferidas$: MillasTransferidas, in $idOrigen$: Id, in $idDestino$: Id,
inout $airMiles$: AirMiles) {
requiere {
$idOrigen \neq idDestino$
$\wedge\, millasTransferidas > 0$
$\wedge\, esUsuarioDe(idDestino, airMiles)$
$\wedge\, esUsuarioDe(idOrigen, airMiles)$
$\wedge_L\, millasDisponibles(airMiles.usuarios[idOrigen]) \geq millasTransferidas$
$\wedge\, A_0 = airMiles$
}
asegura {
$mismaPromo(A_0, airMiles)$
$\wedge\, mismaCantidadDeUsuarios(A_0, airMiles)$
$\wedge\, usuariosIgualesExcepto(A_0, airMiles, \langle idOrigen, idDestino \rangle)$
$\wedge\, esUsuarioDe(idOrigen, airMiles)$
$\wedge\, esUsuarioDe(idDestino, airMiles)$
$\wedge_L\, agregóTransferenciaALosUsuarios(idOrigen, idDestino, millasTransferidas, A_0, airMiles)$
}
}

pred agregóTransferenciaALosUsuarios($idOrigen$: Id, $idDestino$: Id,
$millasTransferidas$: MillasTransferidas, $viejo$: AirMiles, $nuevo$: AirMiles) {
$agregóTransferenciaAlHistorial(viejo.usuarios[idOrigen], nuevo.usuarios[idOrigen],$
$idOrigen, idDestino, -millasTransferidas)$

$\wedge\ agregóTransferenciaAlHistorial(viejo.usuarios[idDestino], nuevo.usuarios[idDestino],$
$idOrigen, idDestino, millasTransferidas)$
}

pred agregóTransferenciaAlHistorial($viejo$: Historial, $nuevo$: Historial, $idOrigen$: Id, $idDestino$: Id, $millasTransferidas$: MillasTransferidas) {
$nuevo.transferencias = viejo.transferencias + +$
$transacciónTransferenciaNueva(idOrigen, idDestino, millasTransferidas)$
$\wedge\ historialesIgualesExceptoTransferencia(viejo, nuevo)$
}

aux transacciónTransferenciaNueva($idOrigen$: Id, $idDestino$: Id, $millasTransferidas$: MillasTransferidas) : TransacciónTransferencia =
$\langle idOrigen : idOrigen,$
$idDestino : idDestino,$
$millasTransferidas : millasTransferidas\rangle$

proc consultarSaldo(in id : Id, in $airMiles$: AirMiles) : Millas {
requiere { $esUsuarioDe(id, airMiles)$ }
asegura { $res = millasDisponibles(airMiles.usuarios[id])$ }
}

proc consultarCategoría(in id : Id, in $airMiles$: AirMiles) : Categoría {
requiere { $esUsuarioDe(id, airMiles)$ }
asegura { $esCategoríaDelUsuario(res, millasTotales(airMiles.usuarios[id]))$ }
}

pred esCategoríaDelUsuario($categoría$: Categoría, millasTotales : Millas) {
$(categoría = Bronce \iff esFactorDe(1, millasTotales))$
$\wedge\ (categoría = Plata \iff esFactorDe(1.5, millasTotales))$
$\wedge\ (categoría = Oro \iff esFactorDe(2, millasTotales))$
$\wedge\ (categoría = Platino \iff esFactorDe(0, millasTotales))$
}

proc rankingMillas(in $cantidadUsuarios$: $\mathbb{Z}$, in $airMiles$: AirMiles) : seq$\langle$Id$\rangle$ {
requiere { $cantidadUsuarios \geq 0$ }
asegura {
$|res| = mínimo(|claves(airMiles.usuarios)|, cantidadUsuarios)$
$\wedge\ cadaUsuarioEstáEn(res, airMiles)$
$\wedge_L\ sonUsuariosConMásMillasDisponibles(res, airMiles)$
$\wedge\ estánOrdenadosPorMayoresMillasDisponibles(res, airMiles)$
$\wedge\ noHayRepetidos(res)$
}
}

aux mínimo($x : \mathbb{Z}, y : \mathbb{Z}$) : $\mathbb{Z}$ = IfThenElse($x < y, x, y$)

pred cadaUsuarioEstáEn($usuarios$: seq$\langle$Id$\rangle$, $airMiles$: AirMiles) {

$(\forall id : \mathsf{Id})\ (id \in usuarios \rightarrow esUsuarioDe(id, airMiles))$
}

pred sonUsuariosConMásMillasDisponibles($usuarios : \mathsf{seq}\langle\mathsf{Id}\rangle, airMiles : \mathsf{AirMiles}$) {
$(\forall mayor, u : \mathsf{Id})$ (
$(mayor \in usuarios \land esUsuarioDe(u, airMiles)) \rightarrow_L$
$(millasDisponibles(airMiles.usuarios[mayor]) \geq millasDisponibles(airMiles.usuarios[u])$
$\lor u \in usuarios)$
)
}

pred estánOrdenadosPorMayoresMillasDisponibles($ids : \mathsf{seq}\langle\mathsf{Id}\rangle, airMiles : \mathsf{AirMiles}$) {
$(\forall i : \mathbb{Z})$ (
$(0 \leq i < |ids| - 1) \rightarrow_L$
$millasDisponibles(airMiles.usuarios[ids[i]]) \geq millasDisponibles(airMiles.usuarios[ids[i+1]])$
)
}

pred noHayRepetidos($ids : \mathsf{seq}\langle\mathsf{Id}\rangle$) {
$(\forall i, j : \mathbb{Z})$ (
$(0 \leq i < |ids| \land 0 \leq j < |ids|) \rightarrow_L$
$ids[i] = ids[j] \iff i = j)$
}

/*Consideramos más informativo mostrar las transferencias recibidas y enviadas por separado en lugar de la suma total. Además, informamos la suma de millas referidas, que son las 500 que se ganan al referir o ser referido por un usuario, sin este dato consideramos que el resumen de transacciones está incompleto.*/

proc resumenTransacciones(in id : Id, in $airMiles$: AirMiles) : $\langle$MillasAcumuladas $\times$ MillasCanjeadas $\times$ MillasTransferidasEnviadas $\times$ MillasTransferidasRecibidas $\times$ MillasReferidas$\rangle$ {
requiere { $esUsuarioDe(id, airMiles)$ }
asegura {
$res = \langle totalMillasAcumuladas(airMiles.usuarios[id]),$
$totalMillasCanjeadas(airMiles.usuarios[id]),$
$totalMillasTransferidasEnviadas(airMiles.usuarios[id]),$
$totalMillasTransferidasRecibidas(airMiles.usuarios[id]),$
$totalMillasReferidas(airMiles.usuarios[id])\rangle$
}
}

proc iniciarPromoción(in $nombre$: string, in $factor : \mathbb{R}$, inout $airMiles$: AirMiles) {
requiere { $factor \geq 1 \land A_0 = airMiles$ }
asegura {
$mismosUsuarios(A_0, airMiles)$
$\land\ airMiles.promo.nombre = nombre$
$\land\ airMiles.promo.factor = factor$
$\land\ airMiles.promo.tope = \langle\rangle$
}
}

```
proc iniciarPromociónConTope(in nombre : string, in factor : ℝ, in tope : ℝ, inout airMiles : AirMiles)
{
   requiere { factor ≥ 1 ∧ tope > 0 ∧ A_0 = airMiles }
   asegura {
        mismosUsuarios(A_0, airMiles)
        ∧ airMiles.promo.nombre = nombre
        ∧ airMiles.promo.factor = factor
        ∧ airMiles.promo.tope = ⟨tope⟩
       }
}

proc finalizarPromoción(inout airMiles : AirMiles) {
   requiere { A_0 = airMiles }
   asegura { mismosUsuarios(A_0, airMiles) ∧ esPromoPredeterminada(airMiles.promo) }
}
```

/*Consideramos que si hay más de un usuario cuyas millas totales sean maximales, podemos devolver cualquiera, ya que esto respetaría lo pedido por la consigna*/

```
proc elQueTieneMásMillas(in airMiles : AirMiles) : Id {
   requiere { hayUsuarios(airMiles) }
   asegura { esUsuarioDe(res, airMiles) ∧_L tieneMásMillasTotales(res, airMiles) }
}

pred tieneMásMillasTotales(u : Id, airMiles : AirMiles)  {
     (∀id : Id) (esUsuarioDe(id, airMiles) →_L
     millasTotales(airMiles.usuarios[u]) ≥ millasTotales(airMiles.usuarios[id]))
}
```

/*Aunque el nombre del proc sea el más canjeador, la consigna pide explícitamente que devuelva el identificador de usuario cuya suma de millas canjeadas y transferidas sea la más alta, o lo que es equivalente, el identificador del usuario que más millas gastó*/ /*Consideramos que si hay más de un usuario que cumpla con este requisito, podemos devolver cualquiera, ya que esto respetaría lo pedido por la consigna*/

```
proc elMásCanjeador(in airMiles : AirMiles) : Id {
   requiere { hayUsuarios(airMiles) }
   asegura { esUsuarioDe(res, airMiles) ∧_L perdióMásMillas(res, airMiles) }
}

pred perdióMásMillas(usuario : Id, airMiles : AirMiles)  {
     (∀u : Id) (
       esUsuarioDe(u, airMiles) →_L
       (millasTotales(airMiles.usuarios[u]) − millasDisponibles(airMiles.usuarios[u]) ≤
       millasTotales(airMiles.usuarios[usuario]) − millasDisponibles(airMiles.usuarios[usuario]))
     )
}
```

/*En nuestro sistema, consideramos que un usuario está definido por su id, por lo que al pedir que se devuelva un usuario nos parece lo más adecuado devolver su número de identificador. También elegimos que el proc devuelva un conjunto y no una secuencia; aunque es lo que define el enunciado, no nos parece

lo más correcto en este caso, ya que el orden de los pares es indistinto*/

```
proc paresTransferidoresExclusivos(in airMiles : AirMiles) : conj⟨⟨Id × Id⟩⟩ {
   requiere { True }
   asegura { tieneTodosLosParesExclusivosOrdenados(res, airMiles) }
}

pred tieneTodosLosParesExclusivosOrdenados(pares : conj⟨⟨Id × Id⟩⟩, airMiles : AirMiles)  {
    (∀u0, u1 : Id) (
      (esUsuarioDe(u0, airMiles) ∧ esUsuarioDe(u1, airMiles)
      ∧L sonTransferidoresExclusivos(u0, u1)) →
      perteneceExactamenteUnaCombinación(u0, u1, pares)
    )
}
```

/*Consideramos que si un usuario no tiene ninguna transferencia, no puede ser el transferidor exclusivo de nadie (de lo contrario, debería ser el transferidor exclusivo de todos los usuarios)*/

```
pred sonTransferidoresExclusivos(u0 : Id, u1 : Id, airMiles : AirMiles)  {
    |airMiles.usuarios[u0].transferencias| > 0
    ∧ |airMiles.usuarios[u1].transferencias| > 0
    ∧ (∀t : TransacciónTransferencia) (
        (t ∈ airMiles.usuarios[u0].transferencias → (t.idOrigen = u1 ∨ t.idDestino = u1))
        ∧(t ∈ airMiles.usuarios[u1].transferencias → (t.idOrigen = u0 ∨ t.idDestino = u0))
      )
}


pred perteneceExactamenteUnaCombinación(u0 : Id, u1 : Id, conjunto : conj⟨⟨Id × Id⟩⟩)  {
    ((u0, u1) ∈ conjunto ∧ (u1, u0) ∉ conjunto)
    ∨ ((u1, u0) ∈ conjunto ∧ (u0, u1) ∉ conjunto)
}
```

2.3 Predicados recurrentes

```
pred esPromoPredeterminada(promo : Promo)  {
    promo.nombre = "NoPromo"
    ∧ promo.factor = 1
    ∧ promo.tope = ⟨⟩
}

pred hayUsuarios(airMiles : AirMiles)  { |claves(airMiles.usuarios)| > 0 }

pred mismaPromo(viejo : AirMiles, nuevo : AirMiles)  { viejo.promo = nuevo.promo }
```

/*Si un diccionario tiene un elemento más que otro, ese elemento no puede tener una clave (identificador) repetida, por propiedad de los diccionarios*/

```
pred tieneUnSoloUsuarioMás(viejo : AirMiles, nuevo : AirMiles)  {
    |claves(viejo.usuarios)| + 1 = |claves(nuevo.usuarios)|
```

}

pred esUsuarioDe(id : Id, $airMiles$: AirMiles) { $id \in claves(airMiles.usuarios)$ }

pred esUsuarioNuevo(id : Id, $viejo$: AirMiles, $nuevo$: AirMiles) {
  $esUsuarioDe(id, nuevo) \land \neg esUsuarioDe(id, viejo)$
}

pred usuariosIgualesExcepto($viejo$: AirMiles, $nuevo$: AirMiles, ids : conj⟨Id⟩) {
  $(\forall id$: Id) (
    $id \notin ids \land esUsuarioDe(id, viejo) \land esUsuarioDe(id, nuevo) \rightarrow_L$
    $viejo.usuarios[id] = nuevo.usuarios[id]$
)
}

pred mismaCantidadDeUsuarios($viejo$: AirMiles, $nuevo$: AirMiles) {
  $|claves(viejo.usuarios)| = |claves(nuevo.usuarios)|$
}

pred esFactorDe($factor : \mathbb{R}, millasTotales$: Millas) {
  $(factor = 1 \iff millasTotales < 10,000)$
  $\land (factor = 1.5 \iff 10,000 \leq millasTotales < 50,000)$
  $\land (factor = 2 \iff 50,000 \leq millasTotales < 1,000,000)$
  $\land (factor = 0 \iff millasTotales \geq 1,000,000)$
}

pred tieneTope($promo : Promo$) { $|promo.tope| = 1$ }

pred mismosUsuarios($viejo$: AirMiles, $nuevo$: AirMiles) { $viejo.usuarios = nuevo.usuarios$ }

pred historialesIgualesExceptoAcumular($viejo$: Historial, $nuevo$: Historial) {
  $viejo.canjes = nuevo.canjes$
  $\land viejo.transferencias = nuevo.transferencias$
  $\land viejo.referencias = nuevo.referencias$
}

pred historialesIgualesExceptoCanjes($viejo$: Historial, $nuevo$: Historial) {
  $viejo.acumular = nuevo.acumular$
  $\land viejo.transferencias = nuevo.transferencias$
  $\land viejo.referencias = nuevo.referencias$
}

pred historialesIgualesExceptoTransferencia($viejo$: Historial, $nuevo$: Historial) {
  $viejo.acumular = nuevo.acumular$
  $\land viejo.canjes = nuevo.canjes$
  $\land viejo.referencias = nuevo.referencias$

}

pred historialesIgualesExceptoReferencia($viejo$: Historial, $nuevo$: Historial) {
$viejo.acumular = nuevo.acumular$
$\wedge\ viejo.canjes = nuevo.canjes$
$\wedge\ viejo.transferencias = nuevo.transferencias$
}

2.4 Auxiliares recurrentes

aux millasTotales($historial$: Historial) : Millas =
$totalMillasAcumuladas(historial)$
$+\ totalMillasReferidas(historial)$
$+\ totalMillasTransferidasRecibidas(historial)$

aux topeDePromoConTope($promo$: Promo) : $\mathbb{R} = \sum_{e \in promo.tope} e$

/*Aunque las millas transferidas enviadas se guardan como negativas, la función auxiliar totalMillas-TransferidasEnviadas devuelve la suma positiva*/

aux millasDisponibles($historial$: Historial) : Millas =
$millasTotales(historial)$
$-\ totalMillasTransferidasEnviadas(historial)$
$-\ totalMillasCanjeadas(historial)$

aux totalMillasAcumuladas($historial$: Historial) : MillasAcumuladas =
$\sum_{i=0}^{|historial.acumular|-1} historial.acumular[i].millasObtenidas$

aux totalMillasCanjeadas($historial$: Historial) : Millas =
$\sum_{i=0}^{|historial.canjes|-1} historial.canjes[i].precioEnMillas$
aux historialVacío() : Historial = $\langle acumular : \langle\rangle, canjes : \langle\rangle, transferencias : \langle\rangle, referencias : \langle\rangle\rangle$

aux totalMillasTransferidasEnviadas($historial$: Historial) : MillasTransferidasEnviadas =
$\sum_{i=0}^{|historial.transferencias|-1}$ IfThenElse($historial.transferencias[i].millasTransferidas < 0$,
$-\ historial.transferencias[i].millasTransferidas, 0$)

aux totalMillasTransferidasRecibidas($historial$: Historial) : MillasTransferidasRecibidas =
$\sum_{i=0}^{|historial.transferencias|-1}$ IfThenElse($historial.transferencias[i].millasTransferidas > 0$,
$historial.transferencias[i].millasTransferidas, 0$)

aux totalMillasReferidas($historial$: Historial) : Millas =
$\sum_{i=0}^{|historial.referencias|-1} historial.referencias[i].millasRecibidas$
}